

NEXAFORGE AI

Adaptive Agent Orchestration Methodology

A production framework for planning, routing, tool execution, recovery, and human oversight in multi-agent systems

Document ID	Version	Owner	Classification
NF-AI-001	2.0	Agent Systems Engineering	Confidential - Internal Use

Purpose

Define how NexaForge AI designs, implements, evaluates, and operates agentic workflows that coordinate language models, tools, deterministic services, and human reviewers. The methodology is intended for systems in which a single prompt-response interaction is insufficient because the task requires decomposition, state, tool use, verification, or controlled escalation.

Keywords: agents, orchestration, routing, planning, tools, LangGraph, recovery, human-in-the-loop

Document Control

Field	Value
Status	Approved methodology baseline
Effective date	21 July 2026
Review cycle	Every 12 months or upon material change
Primary owner	Agent Systems Engineering
Related standards	Security, privacy, software delivery, incident response, and data governance standards
Supersedes	Version 1.0 concise edition

How to Use This Document

Teams should use this document as a design and review guide, not as a substitute for engineering judgment. Sections describe mandatory controls, recommended practices, decision points, and evidence expected at release. Tailoring is permitted when documented and approved by the accountable owner. Any deviation that increases risk must include compensating controls and a review date.

1. Purpose and Objectives

Define how NexaForge AI designs, implements, evaluates, and operates agentic workflows that coordinate language models, tools, deterministic services, and human reviewers. The methodology is intended for systems in which a single prompt-response interaction is insufficient because the task requires decomposition, state, tool use, verification, or controlled escalation.

2. Scope and Applicability

This standard applies to new implementations, major changes, vendor replacements, and material expansions of systems that rely on adaptive agent orchestration methodology. It covers prototypes that process real organizational data, pre-production pilots, and production services. Small experiments using synthetic data may follow a reduced process, but they must still document ownership, purpose, and data handling. The methodology does not replace legal, security, privacy, or domain-specific requirements; instead, it provides the engineering structure through which those requirements are implemented and verified.

3. Core Principles

Explicit state over hidden state: All durable workflow facts are represented in a typed state object. Prompts may interpret state, but they must not become the only place where state exists.

Deterministic control around probabilistic reasoning: Models propose plans and decisions; the runtime enforces budgets, schemas, permissions, retry limits, and transition rules.

Least-capability routing: Use the smallest model, tool set, and autonomy level that can complete the task at the required quality.

Verification before commitment: Actions with business or external effects require validation, policy checks, or approval before execution.

Recoverability by design: Every node defines expected failures, retry behavior, fallback behavior, and a terminal safe state.

4. Lifecycle and Detailed Procedure

1. Intake and task classification

Normalize the request, identify the user objective, detect constraints, classify risk, and determine whether the task is informational, transactional, analytical, or creative. Produce a machine-readable task envelope containing intent, entities, deadlines, sensitivity, and required output format.

- Required evidence: a reviewable artifact or trace showing that intake and task classification was completed.
- Decision record: assumptions, rejected alternatives, owner, and unresolved risks.
- Exit condition: measurable criteria are met before the workflow moves forward.

2. Capability and policy resolution

Resolve which models, tools, data sources, and credentials are permitted. Remove capabilities that are unnecessary for the task. Apply tenant, geography, privacy, and regulatory constraints before planning begins.

- Required evidence: a reviewable artifact or trace showing that capability and policy resolution was completed.
- Decision record: assumptions, rejected alternatives, owner, and unresolved risks.
- Exit condition: measurable criteria are met before the workflow moves forward.

3. Plan construction

Generate a bounded plan represented as ordered or graph-based steps. Each step must specify an owner node, required inputs, expected output schema, success criteria, and maximum cost. Plans are rejected when they contain circular dependencies, unbounded searches, or actions without validation.

- Required evidence: a reviewable artifact or trace showing that plan construction was completed.
- Decision record: assumptions, rejected alternatives, owner, and unresolved risks.
- Exit condition: measurable criteria are met before the workflow moves forward.

4. Dynamic routing

Route each step to a specialist agent or deterministic service using task type, confidence, latency target, data locality, and model availability. Routing decisions are logged with reason codes so they can be audited and improved.

- Required evidence: a reviewable artifact or trace showing that dynamic routing was completed.
- Decision record: assumptions, rejected alternatives, owner, and unresolved risks.
- Exit condition: measurable criteria are met before the workflow moves forward.

5. Tool execution

Call tools through a permissioned gateway that validates arguments, redacts secrets, applies timeouts, and records responses. Tool outputs are treated as untrusted inputs and are schema-validated before entering shared state.

- Required evidence: a reviewable artifact or trace showing that tool execution was completed.
- Decision record: assumptions, rejected alternatives, owner, and unresolved risks.
- Exit condition: measurable criteria are met before the workflow moves forward.

6. Verification and reflection

Verify factual claims, calculations, citations, and action preconditions. Reflection is constrained to specific checks rather than open-ended self-critique. Failed checks generate targeted repair steps.

- Required evidence: a reviewable artifact or trace showing that verification and reflection was completed.
- Decision record: assumptions, rejected alternatives, owner, and unresolved risks.
- Exit condition: measurable criteria are met before the workflow moves forward.

7. Human escalation

Escalate when risk, uncertainty, or policy requires human judgment. The reviewer receives a concise decision packet: original request, proposed action, evidence, uncertainty, and reversible alternatives.

- Required evidence: a reviewable artifact or trace showing that human escalation was completed.
- Decision record: assumptions, rejected alternatives, owner, and unresolved risks.
- Exit condition: measurable criteria are met before the workflow moves forward.

8. Completion and learning

Assemble the final response or execute the approved action. Persist only approved memory, record trace-level metrics, and add failure examples to evaluation sets when they reveal a reusable defect pattern.

- Required evidence: a reviewable artifact or trace showing that completion and learning was completed.
- Decision record: assumptions, rejected alternatives, owner, and unresolved risks.
- Exit condition: measurable criteria are met before the workflow moves forward.

5. Entry Criteria

Work may enter the methodology when a named owner, defined user outcome, initial data classification, and target environment are available. Teams should also identify downstream consumers, irreversible actions, expected traffic, and the cost of wrong answers. If these inputs are unknown, the first deliverable is a discovery brief rather than an implementation.

6. Design Review Expectations

The design review should focus on concrete decisions and evidence. Reviewers inspect architecture diagrams, state or data schemas, model and tool dependencies, evaluation plans, privacy boundaries, fallback behavior, and operating metrics. Open issues must have owners and dates. A design is not approved merely because a model can demonstrate the happy path; the team must show how the system behaves with missing inputs, conflicting evidence, unavailable services, malformed outputs, and unauthorized requests.

7. Documentation and Traceability

Each release maintains a versioned record of requirements, decisions, prompts or policies, model identifiers, data or index versions, test results, known limitations, and approvers. Traceability should allow an incident responder to determine which configuration produced a specific output. Documentation must be concise enough to maintain but detailed enough for a new engineer to reproduce critical behavior.

8. Change Management

Changes are categorized as low, medium, or high impact. Low-impact changes include wording or monitoring adjustments with no behavior change. Medium-impact changes affect routing, prompts, thresholds, or non-sensitive data. High-impact changes include model replacement, new write capabilities, new sensitive data, changed jurisdictions, or altered decision authority. Medium and high-impact changes require targeted regression testing; high-impact changes repeat the relevant risk and approval reviews.

9. Operational Readiness

Before launch, the team confirms monitoring, alert ownership, on-call procedures, rollback, rate limits, access controls, data retention, incident communication, and user support. Synthetic probes should continuously test critical paths. Runbooks should describe both technical recovery and the user-facing behavior during degradation.

10. Continuous Improvement

Production evidence is reviewed on a regular cadence. Teams analyze failures by root cause rather than by surface symptom, distinguish model errors from retrieval, tool, data, policy, or interface errors, and prioritize improvements by user harm and frequency. Confirmed defects become regression tests. Metrics are segmented by use case, language, tenant, and risk tier to avoid hiding poor performance inside global averages.

12. Roles and Responsibilities

Role	Accountability
Product owner	Defines user outcome, service-level objectives, and acceptable autonomy.
Agent architect	Designs graph topology, state schema, routing logic, and recovery paths.
Prompt engineer	Defines node instructions, tool-use constraints, and structured outputs.
Platform engineer	Implements runtime, tool gateway, observability, and deployment controls.
Risk reviewer	Approves high-risk actions, policy mappings, and escalation thresholds.
Evaluation lead	Maintains scenario suites, regression gates, and production scorecards.

13. Measurement Framework

Metric	Definition and Use
Task completion rate	Share of tasks meeting explicit success criteria without manual repair.
Plan validity rate	Percentage of generated plans that pass structural and policy validation.
Tool success rate	Valid tool responses divided by attempted calls, segmented by tool and error type.
Recovery rate	Share of recoverable failures resolved within retry and cost budgets.
Escalation precision	Percentage of escalations that reviewers judge necessary.
Cost per successful task	Total model and tool cost divided by successful completions.
P95 latency	End-to-end latency for the 95th percentile of successful tasks.

Metrics must be segmented by use case and risk tier. Teams should define a baseline, target, alert threshold, and owner for each production metric. A metric without an action rule is considered observational rather than operational.

14. Worked Example

A procurement assistant receives a request to compare three suppliers and draft a recommendation. The intake node classifies the task as analytical with moderate business impact. The planner creates steps for document retrieval, requirement extraction, price normalization, risk scoring, and recommendation drafting. A retrieval agent gathers evidence, a calculation service normalizes currencies, and a policy node checks prohibited supplier attributes. The verifier detects that one supplier quote is expired, causing the graph to request updated evidence rather than fabricate a comparison. The final recommendation includes a confidence label and an approval checkpoint before any purchase order is created.

15. Risk Register and Controls

Risk	Primary Controls
Runaway loops	Use graph-level visit limits, global token and tool budgets, and terminal timeout states.
Tool misuse	Enforce allowlists, argument schemas, read/write separation, and dry-run modes.
State contamination	Namespace state by task and node, validate merges, and prohibit raw prompt text from overwriting trusted fields.
False completion	Require explicit success predicates and evidence checks before marking a task complete.
Over-escalation	Calibrate thresholds using reviewer outcomes and distinguish uncertainty from risk.
Single-agent monoculture	Use deterministic validators and independent evidence channels rather than relying on another model instance alone.

16. Release Gate

- A named business and technical owner has approved the intended use and operating boundaries.
- Required architecture, security, privacy, and risk reviews are complete.
- Evaluation results meet minimum thresholds and no severe unresolved defect remains.
- Monitoring, alerting, rollback, and incident procedures have been tested.
- User-facing disclosures, approval checkpoints, and fallback behavior are implemented where required.
- All model, prompt, tool, data, index, and policy versions are recorded in the release manifest.

17. Review Checklist

- Is the user outcome specific, measurable, and appropriate for AI assistance?
- Are authority boundaries and prohibited actions explicit?

- Can every important output be traced to inputs, evidence, and configuration?
- Does the design handle missing, conflicting, stale, or malicious inputs?
- Are sensitive data, permissions, retention, and deletion controlled?
- Are severe failures represented in the evaluation suite?
- Can the service degrade or stop safely when dependencies fail?
- Are metrics connected to owners and response procedures?

18. Appendices

Typed task envelope example

This appendix should be maintained as a project-specific artifact. It records the concrete implementation details, decisions, evidence, and approvals needed to apply the Adaptive Agent Orchestration Methodology in a reproducible manner.

- Owner and review date
- Inputs and dependencies
- Decision criteria
- Required evidence
- Known limitations and follow-up actions

State transition checklist

This appendix should be maintained as a project-specific artifact. It records the concrete implementation details, decisions, evidence, and approvals needed to apply the Adaptive Agent Orchestration Methodology in a reproducible manner.

- Owner and review date
- Inputs and dependencies
- Decision criteria
- Required evidence
- Known limitations and follow-up actions

Node design template

This appendix should be maintained as a project-specific artifact. It records the concrete implementation details, decisions, evidence, and approvals needed to apply the Adaptive Agent Orchestration Methodology in a reproducible manner.

- Owner and review date
- Inputs and dependencies
- Decision criteria
- Required evidence
- Known limitations and follow-up actions

Incident review questions

This appendix should be maintained as a project-specific artifact. It records the concrete implementation details, decisions, evidence, and approvals needed to apply the Adaptive Agent Orchestration Methodology in a reproducible manner.

- Owner and review date
- Inputs and dependencies
- Decision criteria
- Required evidence
- Known limitations and follow-up actions

Revision History

Version	Date	Summary
1.0	21 July 2026	Initial concise methodology edition.
2.0	21 July 2026	Expanded edition with detailed lifecycle, controls, roles, metrics, examples, risks, release gates, and appendices.